The user sees one world.

How a six-service Minecraft network taught me to design for one boundary — not six JVMs.

10 slides · swipe

Anton Petnitsky · Python / agents / backend

Six JVMs, one login screen.

Velocity (proxy) → Limbo (auth) → Lobby (hub) → Survival, Anarchy, Minigames (game servers). Postgres for permissions. One bot tattling to Telegram. One thing breaks, all of it 'goes down'.

One user. One screen.
Six failure domains.

Anton Petnitsky · Python / agents / backend

The order in which it actually broke.

01 Postgres was up, LuckPerms wasn't ready — no perms anywhere

02 Velocity couldn't forward to the backend — players bounced

03 A stale session.lock blocked the world silently forever

04 Anarchy died, League crashed — cascade of one bad boot order

05 Every one of these looked 'random' to a player

Six services means six places to be wrong.

Anton Petnitsky · Python / agents / backend

Where the boundary belongs.

WRONG

Six boundaries

six ownership models
six monitoring styles
nobody sees the whole

RIGHT

One user boundary

one health view
one alert path
one owner of 'world loads'

Boundaries exist where a user feels the failure — not where org-charts end.

Anton Petnitsky · Python / agents / backend

What the watchdog actually does.

01 pg_isready gate before any Java starts



02 Ping each port; dead JVM gets killed by port, not name



03 Restart in dependency order, never alphabetical



04 Telegram pings me — with the failing layer named

Automation that fixes start order, not one service at a time.

Anton Petnitsky · Python / agents / backend

Your microservices have the same disease.

If one failing dependency cascades silently through three services, you don't have six services — you have one, poorly assembled. Boundary audit, not more retries.

Retries forgive accidents.
Boundaries forgive design.

Anton Petnitsky · Python / agents / backend

The failure that never logs.

Stale session.lock — the world boots, hangs, says nothing. No stack trace, no timeout. Just a silent dead process and a very confusing 'works on my machine' window.

**Silent failures need sensors,
not more logs.**

Anton Petnitsky · Python / agents / backend

MINECRAFT · OBSERVABILITY · 8/10

What I monitor across six services.

01 Each port ping — the only honest health signal

02 Postgres pg_isready — the root of 90% of cascades

03 Dead JVM detection — by port, by PID, never by vibes

04 Telegram alert naming the failing layer — not 'something is wrong'

Alert on the boundary, name the layer, act in order.

Anton Petnitsky · Python / agents / backend

MINECRAFT · THE RULE · 9/10

The architecture rule worth stealing.

Consolidate failure domains where the user feels them — not where your deploy pipeline ends.

One world for the user. Respect the seams invisibly.

Anton Petnitsky · Python / agents / backend

MINECRAFT · KEEP THIS · 10/10

Save this for your next microservice review.

Next time someone proposes splitting a service, ask: who owns the user's journey across it? If nobody answers, the split isn't done — it's just distributed pain.

Save for later review

Anton Petnitsky · Python / agents / backend